CN & SCALER · SESSION 7

Building for Failures

Timeouts, retries, backoff, circuit breakers — and what a video player does when the network lies to it.

EARLY HTTP/1.1 · ASSIGNMENT · DUE BEFORE SESSION 7

Build a calculator that stays on the line

The task

any language, no framework, just a socket

```
GET /add?a=2&b=3    ->  200   5
GET /sub?a=10&b=4   ->  200   6
GET /mul?a=6&b=7    ->  200   42
GET /div?a=9&b=3    ->  200   3

GET /div?a=1&b=0    ->  400
GET /add?a=x&b=3    ->  400
GET /pow?a=2&b=8    ->  404
POST /add           ->  405
GET /add  (no Host)   ->  400
```

That is the entire feature set. You could write it in an afternoon, and the arithmetic is not the point.

Don't read 02-http11/server11.py until you have failed at least once.

How I will mark it

one socket. every request.

```
s = socket.create_connection(("localhost", 8080))

  GET /add?a=2&b=3      -> 200   5
  GET /sub?a=10&b=4     -> 200   6
  GET /mul?a=6&b=7      -> 200   42
  GET /div?a=1&b=0      -> 400
  GET /pow?a=2&b=8      -> 404
  POST /add             -> 405

  socket still open: True
  1 TCP handshake, 6 responses
```

If the socket dies before I am done, you have written a 1996 server.

The part that is actually hard. Keeping the connection open forces a question HTTP/1.0 never asked you: **where does this request end and the next one begin?** While you were hanging up, the answer was "at EOF", for free. Now you must consume exactly Content-Length bytes and not one more — byte n+1 belongs to somebody else.

Stretch, all optional: honour Connection: close · an idle timeout you can defend · chunked encoding · take all six at once and answer in order, which is pipelining.

▸ Now you write the spec

Track 1 — the server

```
$ ./bserve ./www 9000
```

accept a TCP connection
read one binary request frame
map the path to a file under a root
reply: status, headers, the bytes
404 if it is not there
400 if the frame is malformed
and keep the connection open

Track 2 — the client

```
$ ./bcurl -v localhost:9000/index.html
```

build the binary request frame
read the response, body to stdout
-v hexdumps every frame
exit non-zero on 4xx / 5xx

and never open a second connection

The bit in the middle is the actual project

A fixed-size frame header — you pick the fields and the widths, and you defend them. HTTP/2 chose 24 / 8 / 8 / 31. Why?

Headers: number the ten names you actually send, length-prefix the rest — HPACK's first two mechanisms, in an evening.

And one line you may not skip: a receiver meeting a frame type it does not know MUST skip it cleanly. That is how you leave room for a version 2.

What you hand in

1. The spec. Two pages. Enough for a stranger.
2. Your program.
3. An annotated hexdump of one complete request and response.

If you cannot annotate your own bytes, the spec is not finished.

In pairs: one server, one client, and the only thing that crosses between you is the spec. A client that only works against your own server is an implementation, not a protocol.

Upcoming Quiz Schedule

Quiz 1

21th Sep, Monday

Assignment submission

25th Sep, Friday

Today

Quiz 2

12th Oct, Monday

Project submission

8th Oct

Course map — week 7 of 8

WEEK 1
Network programming 101

WEEK 2
Eagle-eye view

WEEK 3
Server-side considerations for scale

WEEK 4
nginx deep dive

WEEK 5
Evolution of HTTP

WEEK 6
(Ab)using CDNs

WEEK 7
Building for failures

WEEK 8
Streaming video at large scale

One discipline: fail on purpose

01	**Recap + failure taxonomy**	basic client/server, HTTP status recap, where a request actually breaks
02	**Is this even a retry?**	the definition that keeps you out of trouble
03	**When to retry**	status codes, idempotency, classify before you act
04	**How to retry**	backoff, jitter, caps, circuit breakers — built live
05	**Case study: a video player's 96 failure modes**	DNS to DRM to iOS ResetMedia
06	**Systems that survive**	failover, degradation, the thundering herd on recovery

echo_server.c

Send it anything. It sends it back, once.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(2026);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 1);

    while (1) {
        int client_fd = accept(server_fd, NULL, NULL);
        char buf[4096];
        int n = read(client_fd, buf, sizeof(buf));
        write(client_fd, buf, n);
        close(client_fd);
    }
}
```

sigpipe_client.c

SO_LINGER {1, 0} sends RST instead of FIN.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    addr.sin_port = htons(8080);
    connect(fd, (struct sockaddr*)&addr, sizeof(addr));

    write(fd, "hello", 5);

    struct linger l = {1, 0}; // abortive close: sends RST instead of FIN
    setsockopt(fd, SOL_SOCKET, SO_LINGER, &l, sizeof(l));
    close(fd);

    return 0;
}
```

01

RECAP + TAXONOMY

What actually breaks, and where

Revise the client/server code and the HTTP status vocabulary from Sessions 1-5 — then place every failure on the timeline of a single request.

callback: Session 1 sockets, Session 5 wire format

One request, six places to fail



- Every failure this session talks about lives on this line. Nothing fails generically — it fails at a specific stage, and the stage decides whether a retry can possibly help.
- A DNS timeout and a read timeout are both “timeouts” in your logs and completely different problems: one never reached a server, the other reached one that might have half-done the work.

callback: Session 5 — HTTP semantics

HTTP status, revisited with one new question

Class	Meaning	What actually happened	Retry?
1xx	Informational	Interim — keep listening, not a final answer	n/a
2xx	Success	The server did the thing	no need
3xx	Redirection	Ask again, somewhere else, maybe with a different method	depends
4xx	Client error	You sent something the server will refuse every time you send it unchanged	usually not
5xx	Server error	The server admits it failed, but says nothing about whether it changed anything	usually yes

- The class taught in Session 5 was what each status means. The question this session adds: does receiving it tell you anything about whether trying again will change the outcome?
- That single question — “will anything be different next time?” — is the entire retry decision. Everything from slide 9 onward is that question applied code by code.

02

DEFINITIONS

Is this actually a retry?

The distinction that separates disciplined systems from ones that quietly get worse under load.

“Retry” has a precise meaning

RETRY

Trying the exact same step again, with nothing changed

Same request. Same method, same body, same headers that matter. You are betting that the SAME action will get a DIFFERENT result, because you believe the failure was transient — the network blinked, the server was briefly overloaded, a load balancer picked an unhealthy backend.

NOT A RETRY

A valid, different action that looks similar

Refreshing an expired token and re-sending the request is NOT a retry — it is a different request (new Authorization header) responding to information the server just gave you. Auth failing outright is usually not something a retry fixes at all: it needs a different credential, not another attempt with the old one.

The test: if you changed nothing about the request and only waited, would you call the second attempt identical to the first? If the answer is no — new token, different endpoint, altered payload — you are not retrying, you are handling.



03

DECISION

When do we retry?

Classify the failure by nature and severity before you touch a retry counter.

Classify before you decide

1	**Where in the request did it fail**	Before a connection existed (DNS, handshake) vs after (mid-response)
2	**Is the server admitting fault**	5xx says “i failed”. 4xx says “you failed”.
3	**Is the action idempotent**	Same effect no matter how many times it runs?
4	**Is this transient or structural**	Will waiting 200ms change anything, or is the resource actually gone?

callback: mindmap — “no point in retrying”

No point retrying

- **404 — the resource is not there**
  - If it doesn't exist now, it won't exist 200ms from now — unless you have specific reason to believe otherwise (see the CDN-propagation exception, slide 22)
- **Before the handshake completes**
  - DNS never resolved, or TCP/TLS never finished — you never reached a server, so nothing about “the server” failed. Retrying the same broken path just burns time twice.
- **Geo-blocks**
  - The request is being refused by policy, not by accident. No amount of retrying changes geography.
- **Preflight failures**
  - The browser or client already decided the real request is not allowed (CORS, mixed content). The real request was never sent.
- **Auth failing outright**
  - Not “don't handle it” — handle it with a token refresh, which is a different, valid flow, not a retry (slide 8).

Worth retrying — nothing was decided against you

SERVER ADMITS FAULT

- 5xx from the server (500 / 502 / 503 / 504)
- Connection timeout — you never got a socket
- Read timeout — you had one, then it went quiet
- DNS server not responding (not “no record” — no answer at all)

ENVIRONMENT SHIFTED UNDER YOU

- Network switch mid-flight (4G → 5G, wifi → cellular)
- CDN edge / load balancer momentarily can't take the request
- Cache not found (an upstream fetch will repopulate it)
- TLS handshake failure on an otherwise-healthy host

callback: RFC 9110 §9.2 — Session 5

The question that decides if blind retry is even safe

Method	Idempotent?	Retry after timeout?	Why
GET	yes	safe, blindly	reading twice changes nothing
PUT	yes	safe, blindly	“set it to X” twice = set to X
DELETE	yes	safe, blindly	already gone → already gone
POST	no	only with an idempotency key	“charge the card” twice = two charges

- You didn't get a response — you don't know if the server finished the work before the connection died.
- For a non-idempotent action, blind retry can double-execute. The fix isn't “don't retry”, it's “make the server able to tell attempt #2 was the same as attempt #1” — an idempotency key sent with the request, checked and deduplicated server-side.

04

MECHANICS

How do we retry?

Build the actual retry loop live — count, backoff, jitter, and the circuit breaker that stops it from making things worse.

The retry loop everyone writes first — and why it's dangerous

```
while True:
    try:
        return call_upstream(request)
    except TimeoutError:
        continue   # try again. immediately. forever.
```

- No max count — a permanently-down dependency becomes an infinite loop, not a fast failure.
- No delay — every client hammers the upstream at wire speed the instant it starts failing.
- No coordination across clients — when the upstream recovers, every retry loop fires on the same tick. This is the thundering herd, and it can re-crash a server that had just come back up.

callback: build this with the class

Exponential backoff, with jitter

```
def retry(fn, max_attempts=5,
          base=0.2, cap=8.0):
    for attempt in range(max_attempts):
        try:
            return fn()
        except Retryable as e:
            if attempt == max_attempts - 1:
                raise
            delay = min(cap, base * 2**attempt)
            delay = random.uniform(0, delay)
            #       ^^^^^^^^^^^^^^^^^^^^^^
            #       "full jitter" — AWS 2015
            time.sleep(delay)
```

THREE KNOBS, THREE JOBS

MAX ATTEMPTS caps total damage — a dead dependency fails in seconds, not forever.

EXPONENTIAL base * 2^attempt spaces retries out so load on the upstream DECAYS instead of holding steady.

FULL JITTER (uniform(0, delay), not delay ± a bit) decorrelates clients. Without it, every client computes the SAME delay and they all retry in lockstep — the herd, just delayed.

When the server tells you exactly how long to wait

Code	Name	What it means	Retry strategy
429	Too Many Requests	Rate limit hit — the server is fine, you're asking too fast	Honor Retry-After header, don't backoff blindly on top of it
503	Service Unavailable	Often paired with Retry-After during planned maintenance / overload shedding	Same — trust the header over your own backoff curve
408	Request Timeout	Server gave up waiting for you to finish sending	Retry is fine — nothing about your request was rejected

- Connection limits and 5/10-second interval caps (mindmap: “how do we retry”) exist for exactly this reason — clients that ignore Retry-After and back off on their own schedule can still cumulatively overwhelm a server that told them, explicitly, to slow down.
- Rule of thumb: a server that hands you a number is smarter about its own load than your exponential curve is. Use the number.

callback: Nygard, Release It! (2007)

The circuit breaker — stop asking a question you already know the answer to

CLOSED

Requests flow through normally. Failures are counted.

→

OPEN

Failure threshold crossed. Requests fail instantly, locally — no call is even attempted.

→

HALF-OPEN

After a cooldown, let ONE request through as a probe. Success → close. Failure → re-open.

- Backoff protects the upstream from ONE client retrying too fast. A circuit breaker protects the upstream from an ENTIRE fleet still calling a service everyone already knows is down — and protects the caller from paying the full timeout cost on every one of those doomed calls.
- This is the max-retry-count idea, one level up: instead of capping retries per request, cap failures per DEPENDENCY, and remember the verdict for a while.

05

CASE STUDY

A video player's 96 reasons to retry

Real production taxonomy, condensed — what changes when the client is a player, not a browser tab.

Why a player's retry logic is harder than an API client's

- **An API client fails visibly — an exception, a 5xx in a log. A player that mishandles a failure fails silently, as a frozen frame or a spinner the user stares at.**
- A single “play” session makes hundreds of requests — manifest, key, and dozens of segments — each with a different retry policy, on one continuous user-visible timeline.
- Some failures aren't network failures at all: a DRM key can be denied, a codec can be unsupported, a decoder can crash — and the correct response often isn't “retry”, it's “fall back to a different stream entirely.”
- The retry budget is a human one: a viewer tolerates a few seconds of rebuffering, not a few minutes of exponential backoff playing out correctly.

callback: mindmap — "network issues"

Network issues — same taxonomy, tighter clock

Failure	What's actually different for a player	Retry?
DNS server not responding	Same as any client — but a player often has a fallback CDN with a different hostname to try instead	retry, or switch CDN
Connection / read timeout	Differential timeouts by request type — a manifest fetch and a segment fetch don't get the same budget	yes, tuned per type
Network switch (4G→5G)	IP address can change mid-download — the in-flight request is dead, not slow	yes, fresh request
Wifi off mid-playback	Buffer keeps playing while offline — retry only needs to start once buffer runs low	yes, deferred
CDN edge not responding	Multi-CDN players fail over to a second CDN rather than retry the same one	switch, don't retry

When 404 doesn't mean “give up”

THE EXCEPTION

404 on a live segment

General rule from slide 11: 404 means don't retry, the resource isn't there. Exception: a live stream's newest segment can 404 for a real reason — the origin is still encoding it, or the CDN hasn't finished propagating it yet. It WILL exist in a few hundred milliseconds.

The fix isn't “retry blindly” — it's a short, bounded retry (2-3 attempts, small fixed delay) specifically for live-edge segment requests, and nowhere else. A 404 on an on-demand asset still means give up.

CUSTOM CODES

419 / 474 / 475 — not in the RFC

Real deployments layer application-specific codes on top of the HTTP registry: 419 (token/session expiry — handle like auth, not a retry) and 474/475 (internal codes seen when a segment is retried in a way that returns a mismatched, partial result).

The lesson isn't the specific numbers — it's that your OWN system's status codes deserve the same classify-before-retry discipline as RFC ones. Document what they mean and whether they're retryable, the same way this deck documented 5xx.

callback: mindmap — “codec/DRM fallback”, “stream issue”

Sometimes the answer isn't retry — it's fall back

DRM key denied / L1 vs L3	Retrying the same license request returns the same denial. Fall back to a lower DRM security level or a different key system.
Decoder / codec unsupported	Not transient — the device genuinely cannot decode this. Fall back to an alternate codec rendition in the manifest.
Master manifest failure → MPD to HLS	If the DASH manifest fails, some players fall back to an HLS rendition of the same content rather than retrying DASH.
Child manifest / segment / init failure	Retryable in the network sense (section 4 backoff applies) — but a repeated failure escalates to a fatal, coded error (e.g. 8018, 8022), not an infinite retry loop.
Ad failure or timeout	Falls back to content, not to a retried ad — the user's time budget for an ad failure is near zero.
iOS ResetMedia	A platform-specific recovery: tear down and rebuild the entire AVPlayer pipeline, because the failure is in device media state, not in a request at all.

Not every retry decision is a network decision

- **Business-rule-based retries**
  - e.g. retry a payment-gated stream start only if the user's entitlement check itself times out — not if it cleanly returns “not entitled.” The HTTP layer succeeded; the business layer made a decision.
- **Change player behavior based on information in the response**
  - The response body, not just the status code, can carry a signal — “degraded, use lower bitrate” or “maintenance, try again in N minutes” — that changes what the player does next far more precisely than a status code alone could.
- **The general lesson**
  - Everything in sections 3-4 classified failures using transport-layer signals (status code, timeout type). Real systems also carry APPLICATION-layer signals, and a mature retry policy reads both.

06

SYSTEMS, NOT REQUESTS

What survives isn't the request — it's the system

Failover, graceful degradation, and the thundering herd that shows up right when everyone thinks the outage is over.

callback: Session 6 — the edge, revisited

49 minutes, and the thing that was still serving traffic

85%
of Fastly's network returning errors

~1 min
to detect the fault

49 min
to 95% recovered — 8 Jun 2021

- Session 6's closing question was: today's edge survives your origin dying, because a cache can serve a stale hit. What does your system do when the edge — or a whole region — is the thing that dies?
- stale-if-error (RFC 5861) is the same idea pointed the other direction: a cache configured to keep serving its last good copy even after it can no longer reach the origin to revalidate it. Not a retry at all — a decision to serve something known-stale rather than nothing.
- The lesson generalizes past caches: a system that can answer with slightly-wrong data survives outages that a system demanding perfectly-fresh-or-nothing does not.

Failover, feature flags, graceful degradation

FAILOVER

Primary → backup

A circuit breaker (slide 18) trips OPEN on the primary and routes to a backup — different region, different provider, different CDN. The client doesn't retry a dead dependency; it stops asking it and asks someone else.

FEATURE FLAGS

Turn off the expensive part

During an incident, a flag can disable a non-essential feature (recommendations, personalization) to shed load off the critical path (playback itself), without a deploy. This is graceful degradation as a lever you can pull in seconds.

GRACEFUL DEGRADATION

Answer smaller, not “no”

Lower bitrate instead of full quality. Cached recommendations instead of live ones. A slightly stale price instead of a spinner. Every case study today has one: the player's own buffer, the CDN's stale-if-error, a lower DRM tier.

callback: slide 16 — jitter, at fleet scale

The second outage: everyone recovering at once

- **The moment a dependency comes back up is the most dangerous moment of the whole incident — not the failure itself.**
- Every client that was retrying, backing off, or waiting on a circuit breaker's cooldown now tries again, and without jitter they tend to be synchronized — they all started failing around the same moment, so their backoff schedules line up.
- The recovering service, at 1% of normal capacity while it warms caches and re-establishes connections, receives close to 100% of normal traffic in the first second it's reachable — and can fall right back over.
- Mitigations: jitter (already have it, slide 16) — staggered circuit-breaker half-open probes rather than all-at-once — and slow-start on the recovering side (accept traffic at a ramped rate, not instantly at full capacity).

One flowchart for every failure this session covered

1. Classify — Where did it fail? Server-admitted (5xx) or client-refused (4xx)? Idempotent?

↓

2. Structural? — 404 / geo-block / preflight / bad auth → stop. Handle, don't retry.

↓

3. Transient → retry — Backoff + full jitter. Cap attempts. Honor Retry-After if given.

↓

4. Dependency-level — Repeated failures trip a circuit breaker — stop asking, fail fast.

↓

5. No good answer left — Fail over, degrade gracefully, or serve stale — anything but nothing.

Things to try

due before Session 8

Wrap a flaky client in a policy, and prove it survives

- **Take a client you already have (the calculator client from Session 5, or any HTTP client) and put it behind a resilient-call wrapper you write from scratch.**
- The wrapper must: classify the failure (retryable vs not, using this session's rules) · back off with full jitter, capped at a max attempt count · honor a Retry-After header when present · open a circuit after N consecutive failures to the same dependency and fail fast while open.
- Point it at a server you also write, one that can be told to misbehave on command: /sometimes-500 (fails a configurable fraction of the time), /always-503-with-retry-after, /always-down (to trigger the circuit breaker).
- Deliverable: the wrapper, the test server, and a short log/plot showing request timestamps under each failure mode — the same kind of evidence Session 6's assignment asked for. Show the backoff curve, the circuit opening, and it closing again after the server recovers.

Five, do at least one

- 1. Take the naive retry loop (slide 15) and the backoff+jitter version (slide 16); simulate 1,000 clients against a rate-limited server and count total requests in the first second under each. Explain the gap.
- 2. Pick three items from the video-player case study (section 5) that aren't retries at all — write one sentence each on what they ARE instead, using the definition from slide 8.
- 3. Design an idempotency key scheme for a POST endpoint of your choice. Where does the server store seen keys, and for how long?
- 4. Read the Fastly postmortem in full (linked in notes) and map its timeline onto the five-step framework (slide 29) — which step, if it had existed, would have shortened the 49 minutes?
- **5. — The one to do if you do only one: instrument a real client you use daily (a CLI tool, a script) that currently has no retry logic. Add classify + backoff + jitter + a cap. Break something on purpose (kill the wifi, block the port) and watch it recover instead of hang.**

the last one

Week 8 — Streaming video at large scale

- Today's retry policy, circuit breaker, and fallback logic were built for a general client. Week 8 puts them inside a real video pipeline: adaptive bitrate, segment prefetch, and the CDN/tiered-cache material from Session 6 all come back.
- The video-player case study (section 5 today) was a preview — next session is where DRM tiers, multi-CDN switching, and manifest fallback stop being examples and become the thing being designed.
- Closing question for this session, to sit with until then: every fallback discussed today — stale cache, backup region, lower bitrate — answers with something WORSE than the ideal. At what point does “worse” stop being acceptable, and who decides?